This shared conversation is generated by AI, for reference only.

good they all codes are working
now provide first four practicals exact code your provided about same to same with proper step by step guide how to do it I am beginner and don't know to use Android studio so guide me in that way

Thought for 9 seconds

Here's a detailed, step-by-step guide for the first four Android practicals using **Kotlin** in Android Studio.
I'll walk you through everything from creating a new project to writing the code and running the app. The code provided is exactly the same as before, so you can copy-paste it.

Before You Start – Setting Up Android Studio

1. **Install Android Studio** (if not already). Download from developer.android.com/studio.
2. **Open Android Studio** and click **"New Project"**.
3. Choose **"Empty Views Activity"** (or "Empty Activity") and click **Next**.
4. Configure your project:
   - **Name**: Give a name (e.g., `CalculatorApp` for Practical 1).
   - **Package name**: Will be auto-filled; you can leave it.
   - **Save location**: Choose a folder.
   - **Language**: Select **Kotlin**.
   - **Minimum SDK**: Choose API 21 or higher (common choice).
5. Click **Finish**. Android Studio will build the project. This may take a minute.

Once the project is open, you'll see two main files we'll work with:

- `MainActivity.kt` (inside `app/src/main/java/yourpackagename/`) – this is where the Kotlin code goes.
- `activity_main.xml` (inside `app/src/main/res/layout/`) – this is where the user interface is designed.

Practical 1: Standard Calculator Application

Step 1: Create a new project

- Follow the steps above and name the project `CalculatorApp`.

Step 2: Design the layout (`activity_main.xml`)

- In the **Project** view (left panel), navigate to **app > res > layout** and double-click `activity_main.xml`.
- You'll see a design preview. Click the **Code** tab at the bottom to switch to XML view.
- **Replace** the entire content with the XML code below. This creates a simple calculator interface with a result display and buttons.

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <TextView
        android:id="@+id/tvResult"
        android:layout_width="match_parent"
        android:layout_height="80dp"
        android:gravity="end"
        android:textSize="24sp"
        android:text="0"
        android:padding="8dp"/>
```

```
<GridLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:columnCount="4"
    android:rowCount="4"
    android:alignmentMode="alignMargins"
    android:columnOrderPreserved="false"
    android:padding="10dp">

    <!-- Row 1 -->
    <Button
        android:id="@+id/btn7"
        android:text="7"
        android:layout_columnWeight="1"
        android:layout_rowWeight="1"
        android:layout_width="0dp"
        android:layout_height="80dp" />
    <Button
        android:id="@+id/btn8"
        android:text="8"
        android:layout_columnWeight="1"
        android:layout_rowWeight="1"
        android:layout_width="0dp"
        android:layout_height="80dp" />
    <Button
        android:id="@+id/btn9"
        android:text="9"
        android:layout_columnWeight="1"
        android:layout_rowWeight="1"
        android:layout_width="0dp"
        android:layout_height="80dp" />
    <Button
        android:id="@+id/btnAdd"
        android:text="+"
        android:layout_columnWeight="1"
        android:layout_rowWeight="1"
        android:layout_width="0dp"
        android:layout_height="80dp" />

    <!-- Row 2 -->
    <Button
        android:id="@+id/btn4"
        android:text="4"
```

```
        android:layout_columnWeight="1"
        android:layout_rowWeight="1"
        android:layout_width="0dp"
        android:layout_height="80dp" />
    <Button
        android:id="@+id/btn5"
        android:text="5"
        android:layout_columnWeight="1"
        android:layout_rowWeight="1"
        android:layout_width="0dp"
        android:layout_height="80dp" />
    <Button
        android:id="@+id/btn6"
        android:text="6"
        android:layout_columnWeight="1"
        android:layout_rowWeight="1"
        android:layout_width="0dp"
        android:layout_height="80dp" />
    <Button
        android:id="@+id/btnSub"
        android:text="-"
        android:layout_columnWeight="1"
        android:layout_rowWeight="1"
        android:layout_width="0dp"
        android:layout_height="80dp" />

    <!-- Row 3 -->
    <Button
        android:id="@+id/btn1"
        android:text="1"
        android:layout_columnWeight="1"
        android:layout_rowWeight="1"
        android:layout_width="0dp"
        android:layout_height="80dp" />
    <Button
        android:id="@+id/btn2"
        android:text="2"
        android:layout_columnWeight="1"
        android:layout_rowWeight="1"
        android:layout_width="0dp"
        android:layout_height="80dp" />
    <Button
        android:id="@+id/btn3"
        android:text="3"
```

```
        android:layout_columnWeight="1"
        android:layout_rowWeight="1"
        android:layout_width="0dp"
        android:layout_height="80dp" />
    <Button
        android:id="@+id/btnMul"
        android:text="*"
        android:layout_columnWeight="1"
        android:layout_rowWeight="1"
        android:layout_width="0dp"
        android:layout_height="80dp" />

    <!-- Row 4 -->
    <Button
        android:id="@+id/btn0"
        android:text="0"
        android:layout_columnWeight="1"
        android:layout_rowWeight="1"
        android:layout_width="0dp"
        android:layout_height="80dp" />
    <Button
        android:id="@+id/btnClear"
        android:text="C"
        android:layout_columnWeight="1"
        android:layout_rowWeight="1"
        android:layout_width="0dp"
        android:layout_height="80dp" />
    <Button
        android:id="@+id/btnEq"
        android:text="="
        android:layout_columnWeight="1"
        android:layout_rowWeight="1"
        android:layout_width="0dp"
        android:layout_height="80dp" />
    <Button
        android:id="@+id/btnDiv"
        android:text="/"
        android:layout_columnWeight="1"
        android:layout_rowWeight="1"
        android:layout_width="0dp"
        android:layout_height="80dp" />
```

```
    </GridLayout>
</LinearLayout>
```

Step 3: Write the Kotlin code (`MainActivity.kt`)

- In the **Project** view, go to **app > src > main > java > yourpackagename** and double-click `MainActivity.kt` .
- **Replace** the existing code with the following:

```kotlin
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.view.View
import android.widget.Button
import android.widget.TextView

class MainActivity : AppCompatActivity() {
    private lateinit var tvResult: TextView
    private var operand1 = 0.0
    private var operand2 = 0.0
    private var operator = ""

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        tvResult = findViewById(R.id.tvResult)
        setNumberClickListeners()
        setOperatorClickListeners()
    }

    private fun setNumberClickListeners() {
        val numberIds = listOf(R.id.btn0, R.id.btn1, R.id.btn2, R.id.btn3, R.
id.btn4,
            R.id.btn5, R.id.btn6, R.id.btn7, R.id.btn8, R.id.btn9)
        numberIds.forEach { id ->
            findViewById<Button>(id).setOnClickListener {
                appendNumber((it as Button).text.toString())
            }
        }
    }
```

```
    private fun appendNumber(number: String) {
        tvResult.append(number)
    }

    private fun setOperatorClickListeners() {
        findViewById<Button>(R.id.btnAdd).setOnClickListener { setOperator
("+") }
        findViewById<Button>(R.id.btnSub).setOnClickListener { setOperator("-
") }
        findViewById<Button>(R.id.btnMul).setOnClickListener { setOperator
("*") }
        findViewById<Button>(R.id.btnDiv).setOnClickListener { setOperator
("/") }
        findViewById<Button>(R.id.btnEq).setOnClickListener { calculate() }
        findViewById<Button>(R.id.btnClear).setOnClickListener { clear() }
    }

    private fun setOperator(op: String) {
        operand1 = tvResult.text.toString().toDouble()
        operator = op
        tvResult.text = ""
    }

    private fun calculate() {
        operand2 = tvResult.text.toString().toDouble()
        val result = when (operator) {
            "+" -> operand1 + operand2
            "-" -> operand1 - operand2
            "*" -> operand1 * operand2
            "/" -> if (operand2 != 0.0) operand1 / operand2 else Double.NaN
            else -> 0.0
        }
        tvResult.text = result.toString()
    }

    private fun clear() {
        tvResult.text = ""
        operand1 = 0.0
        operand2 = 0.0
        operator = ""
    }
}
```

Step 4: Run the app

- Connect an Android device via USB with USB debugging enabled, or create an emulator (AVD).
- Click the green **Run** button (▶) in the toolbar.
- Select your device/emulator and wait for the app to install.
- You can now use the calculator!

Practical 2: Hello World App with Explanation

Step 1: Create a new project

- Name it `HelloWorldApp`.
- Choose **Empty Views Activity** and **Kotlin**.

Step 2: Design the layout (`activity_main.xml`)

- Open `activity_main.xml` and switch to **Code** view.
- Replace the content with:

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World!"
        android:textSize="24sp"/>
</LinearLayout>
```

Step 3: Write the Kotlin code (`MainActivity.kt`)

- Open `MainActivity.kt` and replace with:

```kotlin
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main) // This line links the XML lay
out to this activity
    }
}
```

Step 4: Explanation (to include in your answer/practical)

- `MainActivity` is the **controller**. It inherits from `AppCompatActivity` and manages the app's lifecycle and user interactions. The `onCreate()` method is called when the activity is first created; here we set the content view to our layout.
- `activity_main.xml` is the **user interface definition**. It describes the UI elements (like the TextView) and their arrangement. The XML is "inflated" into actual View objects when `setContentView()` is called.

Step 5: Run the app

- Click **Run** and you'll see "Hello World!" centered on the screen.

Practical 3: Login Screen with Validation

Step 1: Create a new project

- Name it `LoginApp` .

Step 2: Design the layout (`activity_main.xml`)

- Open `activity_main.xml` and replace with:

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <EditText
        android:id="@+id/etUsername"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Username"/>

    <EditText
        android:id="@+id/etPassword"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Password"
        android:inputType="textPassword"/>

    <Button
        android:id="@+id/btnLogin"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Login"
        android:layout_gravity="center_horizontal"/>
</LinearLayout>
```

Step 3: Write the Kotlin code (`MainActivity.kt`)

- Open `MainActivity.kt` and replace with:

```kotlin
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
```

```
import android.widget.Button
import android.widget.EditText
import android.widget.Toast

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val etUsername = findViewById<EditText>(R.id.etUsername)
        val etPassword = findViewById<EditText>(R.id.etPassword)
        val btnLogin = findViewById<Button>(R.id.btnLogin)

        btnLogin.setOnClickListener {
            val username = etUsername.text.toString()
            val password = etPassword.text.toString()

            when {
                username.isEmpty() -> etUsername.error = "Username required"
                password.isEmpty() -> etPassword.error = "Password required"
                else -> Toast.makeText(this, "Login successful", Toast.LENGTH
_SHORT).show()
            }
        }
    }
}
```

Step 4: Run the app

- Click **Run**. Enter something in both fields – if either is empty, an error appears under the field; if both are filled, a toast message “Login successful” appears.

Practical 4: Alert Dialog on Back Button Press

Step 1: Create a new project

- Name it `BackPressDialogApp` .

Step 2: Design the layout (`activity_main.xml`)

- For this practical, any simple layout will do. You can keep the default “Hello World!” TextView, or use the one from Practical 2. I’ll keep it minimal:

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Press back button to see dialog"
        android:textSize="18sp"/>
</LinearLayout>
```

Step 3: Write the Kotlin code (`MainActivity.kt`)

- Open `MainActivity.kt` and replace with:

```kotlin
import androidx.appcompat.app.AlertDialog
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }

    override fun onBackPressed() {
        AlertDialog.Builder(this)
            .setTitle("Exit")
            .setMessage("Are you sure you want to exit?")
            .setPositiveButton("Yes") { _, _ -> finish() }
```

```
            .setNegativeButton("No", null)
            .show()
    }
}
```

Step 4: Run the app

- Click **Run**. When you press the back button (either on the device/emulator's navigation bar or the back arrow in the top bar), an alert dialog appears asking for confirmation.

Summary of Steps for Each Practical

Practical	Project Name	Main Files to Edit
1	CalculatorApp	`activity_main.xml` and `MainActivity.kt`
2	HelloWorldApp	`activity_main.xml` and `MainActivity.kt`
3	LoginApp	`activity_main.xml` and `MainActivity.kt`
4	BackPressDialogApp	`activity_main.xml` and `MainActivity.kt`

For each, you create a new project, replace the XML layout code, replace the Kotlin activity code, and then run.

Tips for Beginners

- **Copy-paste carefully**: Make sure you replace the entire content of each file, not just add to it.

- **Check for red underlines**: If you see errors, double-check that you haven't missed any import statements (Android Studio usually suggests them with Alt+Enter).
- **Run on emulator**: If you don't have a physical device, you can create an emulator (AVD) from the AVD Manager in Android Studio.
- **Save often**: Use `Ctrl+S` (Windows/Linux) or `Cmd+S` (Mac) to save files before running.

If you follow these steps exactly, all four apps should work perfectly. Good luck with your TYBScIT practicals!

now provide next all left practicals with steps

Thought for 11 seconds >

Here are the step-by-step guides for **Practical 5 to Practical 8** using Kotlin in Android Studio. Each guide assumes you know how to create a new project (as shown in the previous answer). I'll explain every file you need to create or modify.

Practical 5: Activity Lifecycle Demonstration

Goal: Log all lifecycle callbacks (`onCreate()` , `onStart()` , etc.) to see when they are called.

Step 1: Create a new project

- Name: `LifecycleApp`
- Template: **Empty Views Activity**
- Language: **Kotlin**

Step 2: Design the layout (`activity_main.xml`)

- Open `app > res > layout > activity_main.xml`
- Switch to **Code** view and replace with a simple message:

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Lifecycle Demo\nCheck Logcat for messages"
        android:textSize="18sp"
        android:gravity="center"/>
</LinearLayout>
```

Step 3: Write the Kotlin code (`MainActivity.kt`)

- Open `app > src > main > java > your.package.name > MainActivity.kt`
- Replace with the code below. It overrides each lifecycle method and prints a log message.

```kotlin
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.util.Log

class MainActivity : AppCompatActivity() {
    private val TAG = "Lifecycle"

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        Log.d(TAG, "onCreate called")
    }

    override fun onStart() {
        super.onStart()
        Log.d(TAG, "onStart called")
    }
```

```
    override fun onResume() {
        super.onResume()
        Log.d(TAG, "onResume called")
    }

    override fun onPause() {
        super.onPause()
        Log.d(TAG, "onPause called")
    }

    override fun onStop() {
        super.onStop()
        Log.d(TAG, "onStop called")
    }

    override fun onDestroy() {
        super.onDestroy()
        Log.d(TAG, "onDestroy called")
    }

    override fun onRestart() {
        super.onRestart()
        Log.d(TAG, "onRestart called")
    }
}
```

Step 4: Run and observe

- Click **Run**.
- Open **Logcat** in Android Studio (bottom panel, usually with a vertical bar). Filter by `Lifecycle`.
- Perform actions: open the app, press home, rotate the screen (if allowed), press back. Watch the log messages appear.

Explanation of each callback (to include in your answer):

- **onCreate()** – Called when the activity is first created. Use for one-time setup (e.g., inflating layout).

- **onStart()** – Called when the activity becomes visible to the user.
- **onResume()** – Called when the activity starts interacting with the user.
- **onPause()** – Called when another activity gains focus; save unsaved data here.
- **onStop()** – Called when the activity is no longer visible.
- **onDestroy()** – Called before the activity is destroyed.
- **onRestart()** – Called after onStop() if the activity is being restarted.

Practical 6: Pass Data from One Activity to Another Using Intent

Goal: First activity sends user input to second activity via `putExtra()` and displays it.

Step 1: Create a new project

- Name: `DataPassApp`
- Template: **Empty Views Activity**
- Language: **Kotlin**

Step 2: Create the second activity

- In the Project view, right-click on your package name under `java`.
- Select **New > Activity > Empty Activity**.
- Name it `SecondActivity`.
- Ensure **Generate Layout File** is checked – it will create `activity_second.xml`.

Step 3: Design the layout for MainActivity (`activity_main.xml`)

- Open `app > res > layout > activity_main.xml` and replace with:

```
xml
```

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <EditText
        android:id="@+id/etData"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter something"/>

    <Button
        android:id="@+id/btnSend"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Send"
        android:layout_gravity="center_horizontal"/>
</LinearLayout>
```

Step 4: Design the layout for SecondActivity (`activity_second.xml`)

- Open `app > res > layout > activity_second.xml` and replace with:

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical">

    <TextView
        android:id="@+id/tvReceived"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="18sp"/>
</LinearLayout>
```

Step 5: Write the code for MainActivity (`MainActivity.kt`)

- Open `MainActivity.kt` and replace with:

```kotlin
import android.content.Intent
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.widget.Button
import android.widget.EditText

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val etData = findViewById<EditText>(R.id.etData)
        val btnSend = findViewById<Button>(R.id.btnSend)

        btnSend.setOnClickListener {
            val data = etData.text.toString()
            val intent = Intent(this, SecondActivity::class.java)
            intent.putExtra("USER_DATA", data)
            startActivity(intent)
        }
    }
}
```

Step 6: Write the code for SecondActivity (`SecondActivity.kt`)

- Open `SecondActivity.kt` (inside the same package) and replace with:

```kotlin
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.widget.TextView

class SecondActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
```

```
        setContentView(R.layout.activity_second)

        val tvReceived = findViewById<TextView>(R.id.tvReceived)
        val receivedData = intent.getStringExtra("USER_DATA")
        tvReceived.text = "Received: $receivedData"
    }
}
```

Step 7: Run the app

- Type something in the EditText, click **Send**. The second activity opens and displays your text.

Practical 7: Custom Color Scheme for Registration Form

Goal: Apply a light background color to the layout and a contrasting color to the submit button.

Step 1: Create a new project

- Name: `RegistrationApp`
- Template: **Empty Views Activity**
- Language: **Kotlin**

Step 2: Design the layout (`activity_main.xml`)

- Open `app > res > layout > activity_main.xml` and replace with:

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp"
    android:background="#E0F7FA">  <!-- Light cyan background -->
```

```
    <EditText
        android:id="@+id/etName"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Name"
        android:layout_marginBottom="8dp"/>

    <EditText
        android:id="@+id/etEmail"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Email"
        android:inputType="textEmailAddress"
        android:layout_marginBottom="16dp"/>

    <Button
        android:id="@+id/btnRegister"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Register"
        android:layout_gravity="center_horizontal"
        android:backgroundTint="#FF5722"/>  <!-- Deep orange accent -->
</LinearLayout>
```

Step 3: Write the Kotlin code (`MainActivity.kt`)

- Open `MainActivity.kt` – we only need to set the layout; no extra logic is required for styling. Replace with:

```kotlin
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
}
```

Step 4: Run the app

- You'll see a registration form with a light cyan background and a deep orange button. (You can change the color codes to any you like.)

Practical 8: BroadcastReceiver for Airplane Mode Changes

Goal: Register a receiver for `Intent.ACTION_AIRPLANE_MODE_CHANGED` and show a Toast when airplane mode is toggled.

Step 1: Create a new project

- Name: `AirplaneModeApp`
- Template: **Empty Views Activity**
- Language: **Kotlin**

Step 2: Design a simple layout (`activity_main.xml`)

- Open `app > res > layout > activity_main.xml` and replace with:

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Airplane Mode Monitor\nToggle airplane mode to see Toast"
        android:textSize="18sp"
        android:gravity="center"/>
</LinearLayout>
```

Step 3: Create the BroadcastReceiver class

- In the Project view, right-click on your package name under `java`.
- Select **New > Kotlin Class/File**.
- Name it `AirplaneModeReceiver`.
- Replace its content with:

```kotlin
import android.content.BroadcastReceiver
import android.content.Context
import android.content.Intent
import android.widget.Toast

class AirplaneModeReceiver : BroadcastReceiver() {
    override fun onReceive(context: Context, intent: Intent) {
        if (intent.action == Intent.ACTION_AIRPLANE_MODE_CHANGED) {
            val isAirplaneModeOn = intent.getBooleanExtra("state", false)
            val message = if (isAirplaneModeOn) "Airplane mode ON" else "Airp
lane mode OFF"
            Toast.makeText(context, message, Toast.LENGTH_SHORT).show()
        }
    }
}
```

Step 4: Register the receiver in `AndroidManifest.xml`

- Open `app > manifests > AndroidManifest.xml`.
- Add the `<receiver>` tag inside the `<application>` section, after the `<activity>` tag. The final manifest should look like this:

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="your.package.name">

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
```

```
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.App">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

        <!-- BroadcastReceiver for airplane mode -->
        <receiver android:name=".AirplaneModeReceiver"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.AIRPLANE_MODE" />
            </intent-filter>
        </receiver>

    </application>
</manifest>
```

Note: The receiver is exported because it needs to receive system broadcasts. The intent filter uses `ACTION_AIRPLANE_MODE` (the constant `Intent.ACTION_AIRPLANE_MODE_CHANGED` corresponds to the string `android.intent.action.AIRPLANE_MODE`).

Step 5: Write minimal code for MainActivity (`MainActivity.kt`)

- Open `MainActivity.kt` and replace with:

```kotlin
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
```

```
    }
}
```

Step 6: Run the app

- Launch the app on a device/emulator.
- Open the quick settings and toggle airplane mode on/off. A Toast message should appear each time.

Summary of Steps for Each Practical

Practical	Project Name	Key Files Created/Modified
5	LifecycleApp	`activity_main.xml`, `MainActivity.kt` (add lifecycle methods)
6	DataPassApp	`activity_main.xml`, `MainActivity.kt`, `activity_second.xml`, `SecondActivity.kt`
7	RegistrationApp	`activity_main.xml` (add colors), `MainActivity.kt` (simple)
8	AirplaneModeApp	`activity_main.xml`, `AirplaneModeReceiver.kt`, `AndroidManifest.xml` (add receiver)

For each new project, always choose **Empty Views Activity** and **Kotlin**. Replace the code as shown, then run.

If you encounter any red underlines in Android Studio, try pressing **Alt+Enter** (Windows/Linux) or **Option+Enter** (Mac) to import missing classes automatically.

Good luck with your practicals!